

Ideal GMSK SNR vs BER Simulation

This directory contains the simulation scripts and modules for computing the Bit Error Rate (BER) across various Signal-to-Noise Ratios (SNR) in an ideal AWGN (Additive White Gaussian Noise) channel for a GMSK-based communication transceiver system.

Files Overview

- **Scripts & Modules:**

- `ideal_snr_vs_ber.m`: The main script that simulates the physical layer loop across different SNRs, computes the Bit Error Rate (BER), and plots the results.
- `build_packet.m`: Prepares and constructs the payload data into the complete burst format (adds preambles, training sequences etc).
- `convEncoder.m` & `viterbi_soft_decoder.m`: Convolutional Encoder (TX) and Viterbi Soft Decoder (RX) for channel coding.
- `GaussianFilter.m`: Applies Gaussian pulse shaping filtering to the digital signal.
- `GmskMapper.m` & `GmskDemapper.m`: GMSK Modulator/Mapper and Demapper respectively.
- `detect_preamble.m`: Computes correlation to detect the burst preamble in the receiving stream.
- `extract_payload.m`: Isolates the payload phase for decoding based on the waveform structure.
- `RxDemodulator.m`: Encapsulates the complete baseband receiver demodulation process.
- `Waveform.m`: Constructs the base structure and calculates symbol counts for bursts/packets (determines lengths of payloads, preambles, and padding).

- **Helper/Data files:**

- `read_axi_stream_csv.m` & `write_axi_stream_csv.m`: Utilities for reading/writing bitstream/bytes from/to CSV files.
- `rx_data.csv` & `tx_data.csv`: Sample CSV files acting as the packet content payload input/output.

Simulation Details

BER Calculation per SNR Value

The Bit Error Rate (BER) calculations are performed using a Monte-Carlo simulation loop over each SNR point:

1. **Prepare Noise Power**: Converts the `SNR_dB` to a linear scale, measures the variance of the pre-computed transmitted signal, and deduces the required noise power (`noisePower = signalPower / SNR_linear`).
2. **Add Channel Noise**: Generates independent Additive White Gaussian Noise (AWGN) possessing the exact noise power calculated, and adds it directly to the clean reference signal.
3. **RX Pipeline**: Feeds the noisy signal backward through the receiver chain:
 - Demodulates the samples to obtain LLRs (Log-Likelihood Ratios).
 - Locates the start of the burst using `detect_preamble.m`. If it can't find a preamble, it considers the entire payload dropped and counts the entire payload size as bit errors.
 - If found, it extracts the target data indices and pushes them through the Viterbi soft decoder.

4. **Compare & Compute:** Compares the original physical info bits stream bit-by-bit against the newly recovered decoded bits.
5. **BER Formula:** Calculates the average bit error rate over all simulated blocks as $BER = \frac{totalBitErrors}{totalBits}$.

Dynamic Bit Limit (How Many Bits are Sent)

The script uses a **dynamic** limit for the number of bits sent per SNR, dictating the duration based on statistical confidence instead of a rigid, fixed amount of bits.

It does this via two guard conditions:

- $MIN_ERRORS = 100$
- $MAX_BITS = 2e6$ (2 million)

For each SNR step, the system stays inside a `while (totalBitErrors < MIN_ERRORS) && (totalBits < MAX_BITS)` loop and repeatedly sends blocks.

- **Low SNRs:** When the signal is weak/noisy, the signal acquires 100 errors in just a handful of packets. The simulation breaks early rather than pointlessly simulating 2 million bits since 100 errors are statistically enough to plot a reliable BER point.
- **High SNRs:** As the SNR gets strong, it takes longer for the noise to cause errors. The script will keep looping until it finally discovers 100 bit errors.
- **Absolute Cap:** If the receiver is experiencing a nearly perfect SNR where errors essentially never occur, waiting for 100 errors could result in an infinite loop. The loop terminates if it reaches the $2e6$ bit limit before hitting 100 errors.